INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE PERNAMBUCO

CAMPUS PAULISTA

CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

VICTOR GIULIANO DE FREITAS SILVA

DIAGNÓSTICO FITOSSANITÁRIO EM FOLHAS DE CANA-DE-AÇÚCAR VIA CNN EMBARCADA COM TINYML, PIPELINE ASSÍNCRONO DUAL-CORE NO ESP32-S3 E CO-SIMULAÇÃO HIL COM VANT

Paulista – PE
2026

VICTOR GIULIANO DE FREITAS SILVA

DIAGNÓSTICO FITOSSANITÁRIO EM FOLHAS DE CANA-DE-AÇÚCAR VIA CNN EMBARCADA COM TINYML, PIPELINE ASSÍNCRONO DUAL-CORE NO ESP32-S3 E CO-SIMULAÇÃO HIL COM VANT

Trabalho de Conclusão de Curso apresentado ao Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia de Pernambuco – Campus Paulista, como requisito parcial para obtenção do título de Tecnólogo em Análise e Desenvolvimento de Sistemas.

Orientador(a): Prof(a). [Nome do Orientador]

Paulista – PE
2026

Sumário

1 Introdução **4**
- 1.1 Contextualização Histórica, Socioeconômica e Relevância Agronômica . . . 4
- 1.2 A Problemática Tecnológica: Conectividade Rural vs. IA em Nuvem 5
- 1.3 A Integração de TinyML com Veículos Aéreos Não Tripulados via MAVLink 5
- 1.4 Objetivos 6
  - 1.4.1 Objetivo Geral 6
  - 1.4.2 Objetivos Específicos 6
- 1.5 Estrutura do Trabalho 7

2 Fundamentação Teórica **8**
- 2.1 Fitopatologia da Cana-de-Açúcar 8
- 2.2 Redes Convolucionais e Arquiteturas Leves (MobileNets) 9
- 2.3 Teoria da Quantização Afim Inteira (INT8) 10
- 2.4 Processamento Digital de Sinais e Teoria de Amostragem na Decodificação de Imagem 11
  - 2.4.1 Espaços de Cor e Viés DC na Conversão RGB565 11
  - 2.4.2 Teorema de Nyquist-Shannon e Aliasing Espacial 12
- 2.5 Arquitetura Computacional do ESP32-S3 e Acelerações SIMD 12
- 2.6 Veículos Aéreos Não Tripulados e Protocolo MAVLink 13

3 Metodologia **15**
- 3.1 Curadoria Criptográfica e Descontaminação do Dataset (MD5) 15
  - 3.1.1 Identificação de Contaminação Cruzada e Duplicação Interna 15
  - 3.1.2 Modelagem Matemática do Colapso da Função de Perda 16
  - 3.1.3 Algoritmo de Descontaminação e Particionamento Físico 16
- 3.2 Arquitetura Neural e Pipeline de Treinamento Otimizado (`v08`) 17
  - 3.2.1 Construção de Grafo Flat para Quantization-Aware Training (QAT) 17
  - 3.2.2 Normalização Externa no Pipeline `tf.data` 17
  - 3.2.3 Cabeça de Classificação com Dual-Pooling 18
  - 3.2.4 Função de Perda Focal e Treinamento em Três Fases 18
- 3.3 Engenharia de Processamento de Imagens e Decodificação no ESP32-S3 . . 18
  - 3.3.1 Decodificação por Blocos MCU via `JPEGDEC` 19
  - 3.3.2 Implementação da Opção C: RGB8888 Nativo + Box Averaging + LUTs em SRAM 19
- 3.4 Otimizações de Baixo Nível, Hierarquia de Memória e SIMD 20
  - 3.4.1 Alocação Híbrida de Memória e Preferred Scratch Buffers em SRAM 20
  - 3.4.2 Requantização Rápida em Assembly SIMD (`CONFIG_NN_SKIP_NUDGE=y`) 20
- 3.5 Arquitetura de Concorrência em Dois Níveis no FreeRTOS 21

3.5.1 Nível 1: Macro-Pipeline Assíncrono com Double Buffering 21
3.5.2 Nível 2: Paralelismo de Dados Intra-Inferência (Fork-Join no `esp-nn`) 21
3.6 Arquitetura do Payload Embarcado e Integração MAVLink com o VANT . 22
3.6.1 Estratégia de Parada Amostral Sistemática (Pause-and-Classify) . . 22
3.6.2 Bancada Experimental de Co-Simulação Hardware-in-the-Loop (HIL) 22

4 Resultados e Discussões **24**
4.1 Resultados da Auditoria Criptográfica MD5 do Dataset 24
4.2 Benchmark de Processamento de Sinais nas 8 Variantes de `JPEGDraw()` . . 25
4.3 Evolução Experimental com o Novo Modelo Neural (`v08`) 25
4.3.1 Desempenho Geral e Comparativo de Acurácia 25
4.3.2 Matriz de Confusão Detalhada (`v08` com Modo 7) 26
4.4 Desempenho no Hardware ESP32-S3: Evolução Arquitetural de Borda . . . 27
4.5 Avaliação Exaustiva em Escala Real (9.269 Imagens) e Análise de Deslocamento de Domínio (*Domain Shift*) 28
4.5.1 Resultados no Dataset Primário Completo (6.748 Imagens) 28
4.5.2 Análise de Deslocamento de Domínio no Dataset Archive (2.521 Imagens) 29
4.6 Validação da Co-Simulação Hardware-in-the-Loop (HIL) com MAVLink . . 29

5 Conclusão **33**
5.1 Síntese das Contribuições 33
5.2 Limitações do Trabalho 34
5.3 Trabalhos Futuros 34

Capítulo 1

Introdução

1.1 Contextualização Histórica, Socioeconômica e Relevância Agronômica

A cultura da cana-de-açúcar (*Saccharum officinarum* L.) constitui um dos pilares históricos, econômicos e socioculturais mais profundos do Brasil e, em particular, do estado de Pernambuco. Desde o período colonial, com a implantação dos primeiros engenhos nas Capitanias Hereditárias no século XVI, a capitania de Pernambuco destacou-se como o polo açucareiro mais próspero da colônia, impulsionando a ocupação demográfica e territorial do litoral e da Zona da Mata nordestina [7]. Conforme documentado pela historiografia clássica brasileira [7, 17], a economia canavieira forjou a estrutura social, a dinâmica política e a fisionomia agrária regional por mais de quatro séculos.

Na contemporaneidade, a despeito da industrialização e diversificação econômica do estado, a cana-de-açúcar preserva centralidade vital na matriz produtiva regional. Dados consolidados da Companhia Nacional de Abastecimento [2, 3] e do Sindicato da Indústria do Açúcar e do Álcool no Estado de Pernambuco [26] atestam que o estado processou aproximadamente 11,6 milhões de toneladas de cana na safra recente, respondendo por parcela substancial do Produto Interno Bruto (PIB) agropecuário nordestino, fornecendo matéria-prima para a produção de açúcar, etanol carburante e bioeletricidade limpa, além de sustentar cerca de 70 mil empregos diretos [24]. Ademais, a cadeia produtiva da cana em Pernambuco abrange milhares de pequenos fornecedores e famílias assentadas na Zona da Mata que dependem economicamente da sanidade de seus canaviais, configurando um ecossistema com forte relevância para a agricultura familiar [20].

Entretanto, a rentabilidade do setor sucroenergético e a subsistência do pequeno produtor enfrentam contínuas perdas agronômicas decorrentes de pragas e fitopatologias foliares de alta transmissibilidade [16]. Dentre os patógenos mais devastadores que acometem os canaviais tropicais, destacam-se o Carvão (*Sporisorium scitamineum*), a Ferrugem Marrom (*Puccinia melanocephala*), o Complexo de *Fusarium* (*Pokkah Boeng*), a Mancha Parda (*Cercospora longipes*), a Podridão de Toletes (*Ceratocystis paradoxa*), a Podridão Vermelha (*Colletotrichum falcatum*) e viroses sistêmicas como a Síndrome da Folha Amarela (SCYLV) e o Mosaico [1]. Essas enfermidades induzem clorose, necroses vasculares nos colmos e quebra drástica no rendimento de sacarose, provocando perdas produtivas de 20% a 50% quando não identificadas precocemente [16, 3].

A contenção fitossanitária eficaz repousa na intervenção localizada (*Precision Spraying*) antes da dispersão das teliósporos e conídios. Contudo, o método tradicional dominante ainda consiste no caminhamento técnico pedestre por pragueiros (*scouting* manual).

Diante do porte denso e cortante da cana adulta (frequentemente superando três metros de altura) e de talhões que ultrapassam centenas de hectares contínuos, a inspeção manual é morosa, excessivamente onerosa, subjetiva e incapaz de monitorar a totalidade das lavouras em tempo hábil [21].

1.2 A Problemática Tecnológica: Conectividade Rural vs. IA em Nuvem

O recente avanço do Aprendizado Profundo (*Deep Learning*) e das Redes Neurais Convolucionais (*Convolutional Neural Networks* – CNNs) viabilizou o diagnóstico automatizado de patologias vegetais a partir de fotos RGB com alta sensibilidade [13, 15]. Todavia, a esmagadora maioria das abordagens disponíveis na literatura acadêmica baseia-se no paradigma da Computação em Nuvem (*Cloud Computing*), no qual imagens adquiridas em campo são enviadas continuamente para servidores remotos providos de placas gráficas de alto custo (GPUs) [28, 22].

No contexto da agricultura brasileira e, em particular, na Zona da Mata pernambucana, a dependência da nuvem esbarra em barreiras intransponíveis:

1. **Ausência de Conectividade Rural:** Grandes porções das zonas rurais sofrem com sombra de sinal de telefonia celular 3G/4G, inviabilizando o upload contínuo de fluxos de vídeo ou pacotes massivos de fotos de alta resolução durante a jornada de inspeção [9];

2. **Latência de Enlace e Incerteza Operacional:** A oscilação dos enlaces sem fio rurais impõe latências imprevisíveis de vários segundos por foto, inviabilizando tomadas de decisão imediatas em controle de voo ou prescrição em tempo real;

3. **Custo de Aceleradores Dedicados:** Computadores de bordo industriais baseados em GPU (como a família NVIDIA Jetson) apresentam custo elevado, peso excessivo e consumo energético incompatível com a realidade orçamentária da agricultura familiar e com aeronaves não tripuladas leves [20].

Como alternativa, a disciplina de *Tiny Machine Learning* (*TinyML*) [27, 18] viabilizou a execução de redes neurais profundas inteiramente quantizadas diretamente em microcontroladores de baixo custo ($<$ US\$5) e consumo na faixa de miliwatts, como o SoC Espressif ESP32-S3 [6]. Ao transferir a inferência matemática para a borda extrema (*Extreme Edge*), elimina-se integralmente a dependência de conectividade externa, garantindo autonomia, determinismo temporal e preservação dos dados.

1.3 A Integração de TinyML com Veículos Aéreos Não Tripulados via MAVLink

Paralelamente, o emprego de Veículos Aéreos Não Tripulados (VANTs), ou drones, revolucionou o sensoriamento remoto agrícola ao conferir elevada mobilidade espacial para inspeção sobre a copa dos canaviais [21].

Entretanto, as abordagens convencionais com drones limitam-se ao processamento estático pós-voo: o VANT sobrevoa a plantação registrando fotos indiscriminadamente em cartões de memória para posterior fotogrametria manual em escritório, horas após

o pouso. Ademais, aeronaves em voo contínuo acopladas a sensores ópticos de baixo custo com obturador eletrônico por rolamento (*rolling shutter*) sofrem severo borrão de movimento (*motion blur*) decorrente da velocidade de translação e das vibrações mecânicas, degradando as feições microscópicas das lesões foliares.

Nesse contexto, surge a oportunidade científica de integrar o microcontrolador ESP32-S3 como um computador de bordo inteligente (*Companion Payload*) acoplado ao piloto automático do drone via protocolo aeroespacial padrão **MAVLink v2** [12]. Em vez de atuar como mero espectador passivo, o microcontrolador executa uma máquina de estados autônoma de **Parada Amostral Sistemática (*Pause-and-Classify*)**: comanda a aeronave a pairar (*hover*) nos waypoints da grade de plantio, aguarda a dissipação de vibrações (500 ms), captura e processa a imagem foliar on-chip em alta resolução, registra a predição sincronizada com a telemetria GPS e libera o voo para o próximo ponto, sintetizando mapas de calor pós-missão para Sistemas de Informação Geográfica (SIG).

1.4 Objetivos

1.4.1 Objetivo Geral

O objetivo geral deste trabalho é projetar, implementar, otimizar e validar experimentalmente um sistema inteligente, autônomo e de baixo custo para avaliação fitossanitária de 11 classes foliares de cana-de-açúcar, composto por uma rede neural convolucional quantizada em inteiros de 8 bits (*INT8*) embarcada no microcontrolador dual-core ESP32-S3, integrada a uma estratégia de controle de VANT via protocolo MAVLink sob o paradigma de Parada Amostral Sistemática (*Pause-and-Classify*).

1.4.2 Objetivos Específicos

Para consecução do objetivo geral, estabelecem-se os seguintes objetivos específicos:

1. Executar uma auditoria forense criptográfica rigorosa baseada em hashes MD5 sobre as 6.748 imagens do *Sugarcane Leaf Image Dataset*, identificando e expurgando duplicatas internas e 614 arquivos envolvidos em colisões cruzadas de rótulos entre classes incompatíveis (notadamente entre *smut* e *Pokkah Boeng*);

2. Desenvolver e treinar o modelo neural MobileNetV2 ($\alpha = 0,75$, entrada 160×160) com Perda Focal Categórica e quantização inteira completa INT8 (`v08`), superando o colapso diagnóstico da classe Carvão;

3. Formular, modelar matematicamente e implementar no firmware C++ o algoritmo de pré-processamento **Modo 7 (RGB8888 + Box-Averaging com LUT Q16 e escala IDCT)**, eliminando o viés DC de escurecimento e garantindo paridade radiométrica exata (PSNR $= \infty$ dB);

4. Projetar uma arquitetura de paralelismo híbrido em dois níveis no FreeRTOS (pipeline assíncrono inter-quadros e *Fork-Join* intra-inferência com alocação de 48 KB de *scratch buffer* na SRAM interna), mitigando a contenção na Octal PSRAM e acelerando as convoluções via instruções vetoriais SIMD Xtensa LX7;

5. Avaliar exaustivamente o sistema no hardware físico sobre **9.269 imagens**, mensurando latência, vazão e diagnosticando empiricamente o fenômeno de Deslocamento de Domínio (*Domain Shift*) frente a sensores heterogêneos;

6. Construir e validar a bancada de Co-Simulação *Hardware-in-the-Loop* (HIL) MAVLink em malha fechada sobre um talhão agrícola de 55 waypoints, sintetizando automaticamente camadas vetoriais GeoJSON (RFC 7946) e mapas de calor de focos epidêmicos compatíveis com QGIS.

1.5 Estrutura do Trabalho

Esta monografia está estruturada em cinco capítulos:

- **Capítulo 2**: Apresenta a fundamentação teórica sobre fitopatologia da cana, redes convolucionais leves (MobileNetV2), quantização inteira simétrica INT8, extensões vetoriais SIMD no ESP32-S3, teoria de amostragem de Nyquist-Shannon e protocolo MAVLink;

- **Capítulo 3**: Descreve a metodologia de engenharia, compreendendo a auditoria MD5 do dataset, o algoritmo de pré-processamento Modo 7, a arquitetura paralela FreeRTOS e a bancada HIL MAVLink;

- **Capítulo 4**: Expõe e discute os resultados experimentais obtidos diretamente no hardware físico, incluindo benchmarks radiométricos, desempenho multinúcleo, avaliação massiva de 9.269 amostras com análise de *Domain Shift* e a missão HIL georreferenciada;

- **Capítulo 5**: Sintetiza as conclusões, avalia o impacto na agricultura familiar e delineia os trabalhos futuros.

Capítulo 2

Fundamentação Teórica

Este capítulo estabelece os alicerces teóricos e matemáticos que sustentam a presente pesquisa, cobrindo a fitopatologia da cana-de-açúcar, arquiteturas convolucionais eficientes, a teoria de quantização afim em inteiros de 8 bits, o processamento de sinais e amostragem na conversão de imagens, a arquitetura de hardware do processador ESP32-S3 e o protocolo aeroespacial MAVLink.

2.1 Fitopatologia da Cana-de-Açúcar

A cana-de-açúcar (*Saccharum officinarum*) é suscetível a uma vasta gama de infecções que afetam a estrutura morfológica e a taxa fotossintética do dossel vegetativo. A caracterização botânica e sintomatológica das 11 classes avaliadas neste trabalho é descrita a seguir:

1. **Carvão da Cana (*smut* — *Sporisorium scitamineum*):** Uma das doenças fúngicas mais graves do mundo. O sintoma patognomônico é a emergência de uma estrutura cilíndrica e chicoteada na gema apical da planta (o "chicote"), recoberta inicialmente por uma membrana prateada que se rompe liberando bilhões de teliósporos marrom-escuros. Causa deformação total das folhas, afinamento dos colmos e perdas de até 100% em variedades suscetíveis.

2. **Pokkah Boeng (*Fusarium moniliforme* / *Gibberella fujikuroi*):** Patologia caracterizada pelo enrugamento, clorose basal e deformação retorcida das folhas apicais jovens (efeito "folha amassada"). Em estágios avançados, o fungo penetra o meristema terminal provocando a podridão do topo da planta e morte do colmo.

3. **Ferrugem Marrom (*BrownRust* — *Puccinia melanocephala*):** Manifesta-se inicialmente por pequenas pontuações cloróticas amareladas visíveis em ambas as faces da folha, que evoluem para pústulas alongadas de coloração marrom-avermelhada paralelas às nervuras. O rompimento da epiderme foliar acelera a dessecação e desfolha precoce.

4. **Mancha Parda (*Brown Spot* — *Cercospora longipes*):** Lesões circulares a ovais de bordas bem delimitadas marrom-escuras com centro avermelhado ou cinza, amplamente disseminadas nas folhas medianas e velhas sob condições de alta umidade.

5. **Clorose Zonada (*Banded Chlorosis*):** Distúrbio fisiológico induzido por estresses térmicos bruscos (ondas repentinas de frio no início do desenvolvimento), caracterizado por faixas horizontais esbranquiçadas ou cloróticas transversais ao longo de todo o limbo foliar.

6. **Síndrome da Folha Amarela (*Yellow Leaf* — *Sugarcane Yellow Leaf Virus* - SCYLV):** Vírus transmitido por pulgões que induz amarelecimento intenso e progressivo da nervura central das folhas intermediárias em direção às margens, inibindo o acúmulo de sacarose e reduzindo o peso do colmo.

7. **Viroses Foliares (*Viral Disease* — Mosaico / *Sugarcane Mosaic Virus*):** Induzem a formação de manchas cloróticas irregulares dispersas em padrão de ilhas verdes escuras contra fundo verde-claro no tecido foliar jovem.

8. **Brotamento Graminoso (*Grassy shoot* — Fitoplasma):** Proliferação desordenada e hipertrofiada de perfilhos finos, cloróticos e desprovidos de colmo aproveitável na base da touceira, conferindo à planta aspecto cespitoso semelhante a uma gramínea.

9. **Podridão Abacaxi do Colmo (*Sett Rot* — *Ceratocystis paradoxa*):** Infecção que acomete prioritariamente os toletes de plantio e a base dos colmos, provocando avermelhamento interno do parênquima com odor aromático frutado característico de ésteres de abacaxi, culminando na podridão negra dos tecidos.

10. **Folhas Secas (*Dried Leaves*):** Senescência avançada e perda completa de turgor celular associada a estresse hídrico severo ou ataques fúngicos terminais.

11. **Folhas Sadias (*Healthy Leaves*):** Tecido foliar íntegro, com pigmentação verde homogênea e ausência de lesões necróticas ou cloróticas.

2.2 Redes Convolucionais e Arquiteturas Leves (MobileNets)

As Redes Neurais Convolucionais tradicionais processam tensores tridimensionais de entrada $X \in \mathbb{R}^{H \times W \times C_{in}}$ através de camadas convolucionais densas compostas por C_{out} filtros de dimensões $K \times K \times C_{in}$. O custo computacional medido em operações de multiplicação e acumulação (*Multiply-Accumulate* - MACs) de uma convolução padrão é expresso por:

$$\text{Custo}_{\text{Padrão}} = H \cdot W \cdot C_{in} \cdot C_{out} \cdot K^2 \tag{2.1}$$

Para microcontroladores de borda dotados de pouca memória e sem aceleradores gráficos, esse custo computacional é proibitivo. A família de arquiteturas MobileNet [10, 23] solucionou esse gargalo fatorando a convolução padrão em duas operações especializadas: a **Convolução em Profundidade** (*Depthwise Convolution*) e a **Convolução Pontual** (*Pointwise Convolution* 1×1), denominadas conjuntamente como **Convoluções Separáveis em Profundidade** (*Depthwise Separable Convolutions*):

1. **Depthwise Convolution ($K \times K$):** Aplica um filtro espacial bidimensional independente a cada canal de entrada, gerando C_{in} mapas intermediários:

$$\text{Custo}_{\text{Depthwise}} = H \cdot W \cdot C_{in} \cdot K^2 \tag{2.2}$$

2. **Pointwise Convolution (1×1):** Realiza uma combinação linear estritamente entre canais ao longo de cada posição espacial para sintetizar as C_{out} características de saída:

$$\text{Custo}_{\text{Pointwise}} = H \cdot W \cdot C_{in} \cdot C_{out} \cdot 1^2 \tag{2.3}$$

A soma dessas duas etapas define o custo total da convolução separável:

$$\text{Custo}_{\text{Separável}} = H \cdot W \cdot C_{in} \cdot K^2 + H \cdot W \cdot C_{in} \cdot C_{out} \tag{2.4}$$

Dividindo a Equação 2.4 pela Equação 2.1, obtém-se a razão exata de redução de esforço computacional:

$$\frac{\text{Custo}_{\text{Separável}}}{\text{Custo}_{\text{Padrão}}} = \frac{H \cdot W \cdot C_{in} \cdot (K^2 + C_{out})}{H \cdot W \cdot C_{in} \cdot C_{out} \cdot K^2} = \frac{1}{C_{out}} + \frac{1}{K^2} \tag{2.5}$$

Para um filtro convencional de $K = 3$ e camadas típicas onde $C_{out} \gg 9$, a razão aproxima-se de $\frac{1}{9} \approx 0,111$. Isso representa uma **redução de cerca de 88% a 90% no volume de cálculos**, tornando a execução viável em frequências de clock de microcontroladores.

A versão MobileNetV2 introduziu ainda os blocos de **Resíduos Invertidos com Gargalos Lineares** (*Inverted Residuals with Linear Bottlenecks*) [23]. Diferente das redes ResNet clássicas que comprimem os canais antes da convolução espacial, os blocos invertidos expandem as características para uma dimensão intermediária maior via projeção 1×1, executam a convolução espacial leve em profundidade e projetam novamente para um gargalo linear estrito de baixa dimensão sem ativação não-linear no ponto de soma residual, preservando a variedade geométrica (*manifold*) das características latentes.

2.3 Teoria da Quantização Afim Inteira (INT8)

A representação de pesos e ativações em precisão de ponto flutuante de 32 bits (*float32*) exige 4 bytes por parâmetro e demanda unidades de ponto flutuante (FPUs) pesadas. A quantização uniforme afim [11] mapeia tensores contínuos reais $r \in [\alpha, \beta]$ para inteiros com sinal de 8 bits $q \in [-128, 127]$ através da transformação matemática:

$$q = \text{clip}\left(\text{round}\left(\frac{r}{S}\right) + Z,\ q_{\min},\ q_{\max}\right) \tag{2.6}$$

onde:

- $S \in \mathbb{R}^+$ representa o fator de escala (*scale factor*), que indica o valor real representado por uma unidade do inteiro quantizado;
- $Z \in \mathbb{Z}$ é o ponto zero (*zero-point*), inteiro exato que mapeia o zero real ($r = 0.0$), garantindo que paddings nulos não sofram distorção numérica;
- $q_{\min} = -128$ e $q_{\max} = 127$ são os limites numéricos do tipo `int8_t`.

A reconstrução aproximada (desquantização) do número real original é dada por:

$$r \approx S \cdot (q - Z) \tag{2.7}$$

Em uma camada convolucional totalmente quantizada onde as ativações de entrada são denotadas por q_1, os pesos do filtro por q_2 e a saída por q_3, a multiplicação tensorial no domínio real $r_3 = r_1 \cdot r_2$ é transcrita para o domínio inteiro como:

$$S_3 \cdot (q_3 - Z_3) = \sum_k \left[S_1 \cdot (q_1^{(k)} - Z_1) \right] \cdot \left[S_2^{(k)} \cdot (q_2^{(k)} - Z_2) \right] \tag{2.8}$$

Isolando a saída quantizada q_3:

$$q_3 = Z_3 + \left(\frac{S_1 \cdot S_2}{S_3} \right) \cdot \sum_k (q_1^{(k)} - Z_1)(q_2^{(k)} - Z_2) \tag{2.9}$$

A constante de escala composta $M = \frac{S_1 \cdot S_2}{S_3} \in (0, 1)$ é decomposta computacionalmente em uma multiplicação inteira de 32 bits com multiplicador normalizado $M_0 \in [0.5, 1.0)$ e um deslocamento de bits à direita (*bit-shift*):

$$M = 2^{-n} \cdot M_0 \tag{2.10}$$

Essa propriedade formalizada por Jacob et al. [11] garante que **100% do laço interno de convolução e acumulação matricial seja executado utilizando exclusivamente acumuladores inteiros de 32 bits e registradores de 8 bits**, sem invocar nenhuma instrução de ponto flutuante em tempo de execução.

2.4 Processamento Digital de Sinais e Teoria de Amostragem na Decodificação de Imagem

2.4.1 Espaços de Cor e Viés DC na Conversão RGB565

Em sistemas embarcados convencionais dotados de decodificadores JPEG, o bitstream Huffman YCbCr é frequentemente convertido para o espaço de cor reduzido **RGB565** (16 bits por pixel: 5 bits para Vermelho, 6 bits para Verde e 5 bits para Azul), visando economizar largura de banda para displays LCD. Para alimentar redes neurais que operam com 8 bits por canal ($[0, 255]$), os programadores comumente aplicam um preenchimento por zeros à direita (*Zero-Fill*):

$$r_8 = r_5 \ll 3 = r_5 \times 8 \tag{2.11}$$

$$g_8 = g_6 \ll 2 = g_6 \times 4 \tag{2.12}$$

$$b_8 = b_5 \ll 3 = b_5 \times 8 \tag{2.13}$$

Demonstra-se matematicamente que esse procedimento introduz duas distorções severas:

1. **Perda de Alcance Dinâmico:** O valor máximo alcançável para r_8 e b_8 é $31 \times 8 = 248$, enquanto para g_8 é $63 \times 4 = 252$. O branco puro (255) torna-se fisicamente inatingível.

2. **Viés DC Negativo Sistemático (*Brightness Bias*):** Sob a hipótese de distribuição uniforme dos bits menos significativos truncados $k \in \{0, \ldots, 2^{8-B} - 1\}$, o erro esperado de amostragem é dado por:

$$E[\epsilon] = -\frac{1}{2^{8-B}} \sum_{k=0}^{2^{8-B}-1} k = -\frac{2^{8-B} - 1}{2} \tag{2.14}$$

Para os canais de 5 bits ($B = 5$, $2^{8-5} - 1 = 7$):

$$\text{Bias}_R = \text{Bias}_B = -\frac{7}{2} = -3,50 \text{ LSB} \tag{2.15}$$

Para o canal de 6 bits ($B = 6$, $2^{8-6} - 1 = 3$):

$$\text{Bias}_G = -\frac{3}{2} = -1,50 \text{ LSB} \tag{2.16}$$

O viés DC médio esperado para imagens RGB é:

$$\text{Bias}_{\text{médio}} = \frac{-3,50 + (-1,50) + (-3,50)}{3} = \mathbf{-2,833} \text{ LSB} \tag{2.17}$$

Esse deslocamento negativo atenua sistematicamente os pixels de toda a imagem, escurecendo folhas verdes e degradando padrões de contraste foliar.

2.4.2 Teorema de Nyquist-Shannon e Aliasing Espacial

O Teorema da Amostragem de Nyquist-Shannon [19, 25] estabelece que um sinal bi-dimensional contínuo $f(x, y)$ com frequência espacial máxima $(u_{\max}, v_{\max})$ só pode ser perfeitamente reconstruído sem distorção se a frequência de amostragem satisfizer:

$$f_{s,x} \geq 2u_{\max}, \quad f_{s,y} \geq 2v_{\max} \tag{2.18}$$

Ao reduzir a resolução espacial de uma imagem de entrada de dimensões $W_{orig} \times H_{orig}$ (768×1024) para o tensor de entrada da CNN de dimensões $W_{dst} \times H_{dst}$ (160×160), a taxa de decimação é de aproximadamente $4,8 \times 6,4 \approx 30,72$ pixels por elemento de destino.

Se o processo de subamostragem for implementado por decimação ingênua (amostragem pontual por sobrescrita ou vizinho mais próximo), as frequências espaciais superiores à frequência de Nyquist da grade de destino sofrem rebatimento espectral (*aliasing*), projetando componentes espaciais de alta frequência sobre as baixas frequências [8]. Fisicamente, lesões finas de esporos fúngicos com espessura de 1 a 3 pixels na imagem original desaparecem ou assumem formatos geométricos artificiais.

Para cumprir o critério de Nyquist, deve-se aplicar previamente um filtro passa-baixas anti-aliasing sobre cada bloco de decimação antes de realizar a amostragem. No contexto computacional discreto de imagens, o integrador ideal de área é a Média de Caixa (*Box Averaging*):

$$I_{dst}(x', y') = \frac{1}{|\Omega(x', y')|} \sum_{(x,y) \in \Omega(x', y')} I_{orig}(x, y) \tag{2.19}$$

onde $\Omega(x', y')$ representa o conjunto de coordenadas espaciais da imagem de alta resolução que mapeiam geometricamente para a célula discreta (x', y') do tensor de entrada.

2.5 Arquitetura Computacional do ESP32-S3 e Acelerações SIMD

O microcontrolador Espressif ESP32-S3 incorpora dois núcleos de 32 bits baseados na arquitetura Xtensa LX7 operando em frequência de clock de até 240 MHz [5]. Para cargas

de trabalho de Inteligência Artificial, o chip inclui a extensão proprietária **PIE** (*Processor Instruction Extensions*), dotada de 8 registradores vetoriais dedicados de 128 bits (`q0` a `q7`) e acumuladores de 160 bits (`accx`, `qacc`).

Através das instruções SIMD vetoriais, como `ee.vld.128.ip` (carga alinhada de 16 bytes com auto-incremento de ponteiro) e `ee.vmulas.s8.accx` (multiplicação-acumulação vetorial saturada), o processador é capaz de executar **16 operações MAC inteiras de 8 bits em um único ciclo de máquina**.

Em termos de hierarquia de memória, o microcontrolador organiza-se em dois domínios principais:

- **SRAM Interna (512 KB totais, 309 KB dinâmicos livres no boot):** Opera na mesma frequência da CPU com zero estados de espera (*zero wait-states*), oferecendo acesso contíguo de 128 bits para o motor vetorial SIMD e canais DMA;
- **Octal PSRAM Externa (8 MB):** Memória estática externa acoplada por barramento serial síncrono Octal SPI a 40/80 MHz. Embora ofereça ampla capacidade de armazenamento, compartilha a cache L1 de dados (DCache), sofrendo penalidades de latência de barramento sempre que ocorrem falhas de cache (*cache misses*).

O gerenciamento multitarefa do firmware é provido pelo sistema operacional de tempo real FreeRTOS, permitindo a fixação de tarefas por afinidade de núcleo (*Task Pinning*) e comunicação assíncrona orientada a eventos através de semáforos e filas de mensagens (*Queues*).

2.6 Veículos Aéreos Não Tripulados e Protocolo MAVLink

O protocolo MAVLink (*Micro Air Vehicle Link*) [12] é uma especificação padrão de comunicação serial binária ultraleve amplamente utilizada no ecossistema aeroespacial de drones autônomos de código aberto (projetos ArduPilot e PX4).

O pacote básico do MAVLink versão 2.0 possui sobrecarga (*overhead*) de apenas 14 a 27 bytes, incluindo delimitador de início (`STX = 0xFD`), tamanho do payload, flags de compatibilidade, contadores de sequência, ID do sistema (`sysid`), ID do componente (`compid`), ID da mensagem (24 bits), carga útil de dados e soma de verificação ciclíca de redundância CRC-16.

Dentre os principais modos de voo autônomos suportados pelos pilotos automáticos, o modo **GUIDED** (Guiado) permite que um computador de bordo secundário (*Companion Computer*) assuma o controle da rota dinâmica do veículo sem intervenção humana, emitindo comandos de navegação tridimensionais via porta serial UART:

- `SET_POSITION_TARGET_GLOBAL_INT`: Comanda o deslocamento do drone para coordenadas geográficas arbitrárias (Latitude, Longitude e Altitude) com controle estrito de velocidade e aceleração;
- `MAV_CMD_DO_PAUSE_CONTINUE`: Permite pausar momentaneamente a execução de um plano de voo autônomo, forçando o drone a manter estabilização estática de pairamento (*hovering* / `LOITER`) contra ventos externos;

- `GLOBAL_POSITION_INT`: Mensagem de telemetria transmitida periodicamente pelo drone a frequências entre 4 Hz e 10 Hz, contendo coordenadas GPS filtradas por fusão sensorial (Filtro de Kalman Estendido - EKF), altitude barométrica e velocidades relativas nos eixos norte, leste e descida.

Capítulo 3

Metodologia

Este capítulo descreve a metodologia de engenharia aplicada no desenvolvimento do sistema, abrangendo a curadoria criptográfica do dataset, o pipeline de treinamento da rede neural quantizada (v08), a teoria e implementação do pré-processamento de imagens no firmware C++, a hierarquia de memória e acelerações SIMD no microcontrolador ESP32-S3, a arquitetura de concorrência FreeRTOS e a integração de controle autônomo de voo via protocolo MAVLink em bancada de co-simulação *Hardware-in-the-Loop* (HIL).

3.1 Curadoria Criptográfica e Descontaminação do Dataset (MD5)

O ponto de partida do trabalho envolveu o *Sugarcane Leaf Image Dataset*, um conjunto público contendo 6.748 imagens em formato JPEG com resoluções típicas de 768×1024 pixels, catalogadas originalmente nas 11 classes patológicas descritas na Seção 2.1.

3.1.1 Identificação de Contaminação Cruzada e Duplicação Interna

Durante as primeiras rodadas experimentais com o modelo de linha de base (`v07`), observou-se que a classe *smut* (Carvão) apresentava taxa de acerto inferior a 6,0%, com confusão sistemática de predição em relação à classe *Pokkah Boeng*. Para investigar a causa raiz do problema, desenvolveu-se um script de auditoria em Python que calculou o hash criptográfico MD5 (*Message Digest Algorithm 5*) para todos os 6.748 arquivos binários da base.

A varredura exaustiva revelou duas anomalias estruturais severas no dataset original:

1. **Duplicação Interna (*Data Leakage* — 747 arquivos):** Presença de centenas de fotos idênticas repetidas dentro da mesma pasta de classe (com destaque para 288 duplicatas em *Brown Spot* e 276 em *Yellow Leaf*). Em uma divisão aleatória convencional de dados (`validation_split=0.20`), réplicas exatas da mesma folha caíam simultaneamente nos conjuntos de treino e teste, mascarando o sobreajuste (*overfitting*).

2. **Contaminação Cruzada de Rótulos (*Cross-Class Label Collision* — 614 arquivos / 306 pares):** Descoberta de 306 pares de imagens binariamente idênticas (mesmo hash MD5, byte a byte) arquivadas simultaneamente sob pastas de patologias

distintas e incompatíveis. O caso mais crítico concentrou-se entre as classes *Pokkah Boeng* e *smut*, com **110 arquivos idênticos** rotulados com ambas as doenças.

3.1.2 Modelagem Matemática do Colapso da Função de Perda

A presença de dados idênticos com rótulos contraditórios afeta negativamente o treinamento por gradiente descendente. Sejam duas classes conflitantes A (*Pokkah Boeng*) e B (*smut*). Durante o treinamento supervisionado, a mesma imagem de entrada x é apresentada à rede associada ao vetor *one-hot* $y_A = [1, 0]^T$ sob um peso de classe $w_A = 1,44$, e em outro instante associada a $y_B = [0, 1]^T$ sob peso $w_B = 1,39$.

A função de perda por Entropia Cruzada Categórica ponderada sobre essa amostra x é expressa por:

$$\mathcal{L}(x) = -w_A \log p_A(x) - w_B \log p_B(x) \tag{3.1}$$

Como a camada final Softmax impõe a restrição $p_A(x) + p_B(x) = 1$ (assumindo probabilidade desprezível para as demais classes), pode-se reescrever a Equação 3.1 como uma função univariada de p_A:

$$\mathcal{L}(p_A) = -w_A \log p_A - w_B \log(1 - p_A) \tag{3.2}$$

Para encontrar o ponto de convergência ótimo do otimizador, calcula-se a primeira derivada e iguala-se a zero:

$$\frac{d\mathcal{L}}{dp_A} = -\frac{w_A}{p_A} + \frac{w_B}{1 - p_A} = 0 \implies w_A(1 - p_A) = w_B p_A \tag{3.3}$$

Isolando p_A^*:

$$p_A^* = \frac{w_A}{w_A + w_B} \tag{3.4}$$

Substituindo os pesos empíricos do treinamento ($w_A = 1,44$ e $w_B = 1,39$):

$$p^*_{\text{Pokkah}} = \frac{1,44}{1,44 + 1,39} = \mathbf{50,88\%}, \quad p^*_{\text{smut}} = \frac{1,39}{1,44 + 1,39} = \mathbf{49,12\%} \tag{3.5}$$

Essa dedução matemática comprova analiticamente por que o modelo `v07` estabilizava sua confiança média em torno de $\approx 53\%$ e errava sistematicamente a classe *smut*: como a probabilidade ótima de *Pokkah Boeng* era ligeiramente superior ($50,88\% > 49,12\%$), a função argmax predizia *Pokkah Boeng* em todas as folhas de teste onde a contaminação estava presente.

3.1.3 Algoritmo de Descontaminação e Particionamento Físico

Para erradicar o problema, implementou-se a rotina automatizada `construir_dataset_limpo_estratif` na Célula 5 do notebook `notebooks/cane_disease_esp32s3_v08.ipynb`. O algoritmo opera em três etapas:

1. Calcula o hash MD5 de todas as 6.748 imagens e constrói um índice invertido `hash -> lista_de_caminhos`;

2. Identifica e expurga as 747 duplicatas internas e os 614 arquivos envolvidos em colisões cruzadas, preservando exatamente **5.389 imagens 100% únicas e consistentes**;

3. Constrói árvores de diretórios físicos no disco (`train/` e `val/`) através de links simbólicos (*symlinks*), aplicando partição estratificada de **80% para treino (4.313 imagens)** e **20% para validação (1.076 imagens)** com preservação rigorosa da proporção de classes.

3.2 Arquitetura Neural e Pipeline de Treinamento Otimizado (`v08`)

A concepção da rede neural no notebook `v08` baseou-se na arquitetura MobileNetV2 ($\alpha = 0,35$, resolução de entrada $160 \times 160 \times 3$) adaptada para atender às restrições estritas de alocação de memória do microcontrolador.

3.2.1 Construção de Grafo Flat para Quantization-Aware Training (QAT)

No framework Keras tradicional, instanciar um modelo pré-treinado através de chamadas funcionais aninhadas (`base_model(inputs)`) encapsula o grafo interno, impedindo que o módulo de quantização oficial `tensorflow_model_optimization` (TFMOT) insira emuladores de quantização (*FakeQuant*) nas camadas intermediárias.

Para sanar essa limitação, a arquitetura foi montada explicitamente como um **Grafo Plano** (*Flat Graph*):

```
inputs = layers.Input(shape=(160, 160, 3), name="input_tensor")
base_model = tf.keras.applications.MobileNetV2(
    input_tensor=inputs,
    alpha=0.35,
    include_top=False,
    weights="imagenet"
)
```

Essa formulação expõe todos os nós convolucionais diretamente no grafo raiz, permitindo que a função `tfmot.quantization.keras.quantize_model()` insira operadores *FakeQuant* em **100% das 52 camadas convolucionais** da rede.

3.2.2 Normalização Externa no Pipeline `tf.data`

A inclusão de camadas de reescalonamento (`layers.Rescaling(1./127.5, offset=-1)`) dentro do modelo Keras frequentemente induz o conversor TFLite a inserir nós desnecessários de conversão de tipos (`DEQUANTIZE` e `QUANTIZE`) na entrada da rede. No ESP32-S3, essa conversão intermediária em ponto flutuante exigiria um buffer temporário de $160 \times 160 \times 3 \times 4$ bytes = **307,2** KB na memória RAM.

A normalização foi transferida integralmente para a camada externa de dados do `tf.data`:

$$x_{\text{norm}} = \frac{x_{\text{uint8}}}{127,5} - 1,0 \tag{3.6}$$

Com isso, o modelo TFLite exportado opera com entrada nativa puramente inteira `int8_t` com parâmetros $S = \frac{1}{127,5} \approx 0,007843138$ e $Z = 0$, casando perfeitamente com a subtração direta `pixel_u8 - 128` implementada em C++ no microcontrolador.

3.2.3 Cabeça de Classificação com Dual-Pooling

Em patologias foliares de cana-de-açúcar, algumas doenças caracterizam-se por manchas necróticas puntiformes e concentradas (*Brown Spot*), enquanto outras provocam clorose ampla e homogênea (*Banded Chlorosis*). O uso isolado de `GlobalAveragePooling2D` tende a suavizar e diluir feições pequenas, enquanto o `GlobalMaxPooling2D` ignora o contexto global da folha.

A cabeça de classificação foi projetada combinando ambas as operações em paralelo (*Dual-Pooling*):

```
avg_pool = layers.GlobalAveragePooling2D(name="avg_pool")(x)
max_pool = layers.GlobalMaxPooling2D(name="max_pool")(x)
concat = layers.Concatenate(name="dual_pool")([avg_pool, max_pool])
dropout = layers.Dropout(0.30, name="top_dropout")(concat)
outputs = layers.Dense(11, activation="softmax", name="predictions")(
    dropout)
```

3.2.4 Função de Perda Focal e Treinamento em Três Fases

Para balancear a dificuldade de separação entre classes com sintomas visuais parecidos, adotou-se a função de perda *Categorical Focal Crossentropy* [14] com parâmetro modulador de foco $\gamma = 2,0$, suavização de rótulos (*label smoothing*) de $0,02$ e pesos de classe atenuados por potência de três quartos:

$$w_c = \left(\frac{N_{\text{total}}}{11 \cdot N_c} \right)^{0,75} \tag{3.7}$$

O regime de treinamento foi executado em três fases complementares:

- **Fase 1 (Aquecimento do Head):** 15 épocas com o backbone congelado utilizando otimizador Adam com taxa de aprendizado inicial $\eta = 10^{-3}$;
- **Fase 2 (Fine-Tuning com BatchNorm Congelado):** Descongelamento dos pesos convolucionais com $\eta = 5 \times 10^{-5}$, forçando explicitamente `layer.trainable = False` para todas as camadas de `BatchNormalization`. Isso evitou a corrupção das médias e variâncias móveis acumuladas do ImageNet durante a retropropagação;
- **Fase 3 (QAT com Calibração Protegida):** Aplicação de `quantize_model()` com $\eta = 10^{-5}$ e `EarlyStopping(monitor='val_loss', patience=6, start_from_epoch=4)`. A restrição `start_from_epoch=4` impediu que o mecanismo de parada antecipada abortasse o treinamento durante as flutuações iniciais das faixas dinâmicas de 8 bits.

O modelo quantizado final foi exportado no arquivo `data/model_mbnet_int8.tflite`, ocupando apenas **639.272 bytes** (624 KB).

3.3 Engenharia de Processamento de Imagens e Decodificação no ESP32-S3

A decodificação JPEG e a redução espacial para o tensor $160 \times 160 \times 3$ foram implementadas no módulo `src/image_processor.cpp`.

3.3.1 Decodificação por Blocos MCU via `JPEGDEC`

Descomprimir previamente a foto bruta de 768×1024 em um framebuffer RGB888 intermediário exigiria $768 \times 1024 \times 3 = \mathbf{2,35}$ MB de memória RAM contígua. Como a SRAM interna do ESP32-S3 dispõe de 309 KB livres e a PSRAM opera sob barramento compartilhado, essa abordagem inviabilizaria o sistema.

Utilizou-se a biblioteca `JPEGDEC`, que decodifica o fluxo JPEG linha a linha em blocos MCU (*Minimum Coded Units*, tipicamente 16×16 pixels) mantendo um buffer estático interno de apenas **4 KB** (2.048 palavras de 16 bits). A cada bloco decodificado, a biblioteca aciona o callback de desenho customizado `JPEGDraw(JPEGDRAW *pDraw)`.

3.3.2 Implementação da Opção C: RGB8888 Nativo + Box Averaging + LUTs em SRAM

Para solucionar simultaneamente o viés DC de escurecimento (-2,90 LSB) comprovado na Seção 2.4.1 e o aliasing espacial decorrente da decimação pontual, o firmware implementou a **Opção C (Modo 7)**, estruturada em três pilares de engenharia:

1. **Extração Direta em 24 bits (`RGB8888`):** Configuração de `jpeg.setPixelType(RGB8888)`, ativando a rotina interna `JPEGPixel2RGB()` que converte o domínio YCbCr diretamente para amostras de 32 bits Little Endian (`0xFFBBGGRR`, 8 bits puros por canal), sem jamais degradar a representação cromática para 16 bits (`RGB565`);

2. **Pré-cálculo de LUTs Espaciais em SRAM Interna:** Para converter coordenadas da imagem de entrada (src_x, src_y) para as coordenadas do tensor (dst_x, dst_y), uma rotina ingênua executaria duas divisões inteiras por pixel:

$$dst_x = \frac{src_x \cdot 160}{W_{orig}}, \quad dst_y = \frac{src_y \cdot 160}{H_{orig}} \tag{3.8}$$

   Em uma imagem de 768×1024, isso representaria $786.432 \times 2 = 1.572.864$ operações de divisão por quadro. A função `build_src_to_dst_lut()` pré-computa dois vetores estáticos em SRAM interna (`s_src_to_dst_x[1024]` e `s_src_to_dst_y[1024]`), reduzindo o custo de mapeamento de $1,57$ milhão de divisões para apenas **1.792 divisões de inicialização por foto**, transformando o loop interno em acessos de vetor de 1 ciclo de clock;

3. **Acumulador Espacial Alinhado e Média Aritmética:** Alocou-se na Octal PSRAM uma matriz contígua alinhada em 16 bytes de estruturas `BoxAccumulator` com $160 \times 160 = 25.600$ células:

```
struct alignas(16) BoxAccumulator {
    uint32_t sum_r;
    uint32_t sum_g;
    uint32_t sum_b;
    uint16_t count;
};
```

   A cada pixel processado, o callback acumula os componentes de 8 bits e incrementa o contador. Na finalização do frame, a função `finalize_box_average_to_tensor()`

calcula a média aritmética simetricamente arredondada ao inteiro mais próximo e aplica a quantização afim:

$$r = \left\lfloor \frac{\text{sum_r} + (\text{count} \gg 1)}{\text{count}} \right\rfloor \tag{3.9}$$

$$\text{tensor}[i] = \text{int8_t}(r - 128) \tag{3.10}$$

3.4 Otimizações de Baixo Nível, Hierarquia de Memória e SIMD

A execução eficiente da rede neural quantizada no ESP32-S3 dependeu de intervenções específicas na arquitetura de memória e nos kernels vetoriais da biblioteca `esp-nn` [4].

3.4.1 Alocação Híbrida de Memória e Preferred Scratch Buffers em SRAM

Inspecionando os tensores da MobileNetV2 quantizada, constatou-se que o pico de consumo de mapas de ativação intermediários (*ping-pong*) nas camadas convolucionais iniciais atinge **409 KB** (Tensor 13 de 300 KB somado ao Tensor 10 de 100 KB). Esse volume excede a SRAM interna livre do microcontrolador (309 KB no boot).

Portanto, a `tensor_arena` de 800 KB e o buffer do modelo FlatBuffer (624 KB) foram obrigatoriamente alocados na Octal PSRAM externa via `heap_caps_malloc(..., MALLOC_CAP_SPIRAM)`.

Todavia, identificou-se que a alocação dos *workspaces* temporários de convolução (*im2col*, transposição e alinhamento SIMD) na PSRAM externa degradava o desempenho por contenção no barramento Octal SPI. Para resolver esse gargalo:

1. Alocou-se estaticamente no boot um buffer contíguo de **48 KB** (`s_sram_scratch_buf`) na **SRAM Interna rápida com suporte a DMA** (`MALLOC_CAP_INTERNAL | MALLOC_CAP_DMA`, alinhado em 16 bytes);

2. Registrou-se esse ponteiro através das chamadas de API da Espressif `esp_nn_set_conv_preferred_` e `esp_nn_set_depthwise_conv_preferred_scratch_buf_esp32s3()`;

3. Aplicou-se um patch cirúrgico no arquivo `esp_nn_conv_esp32s3.c` da biblioteca oficial, estendendo o uso do buffer interno também para a camada de entrada (3×3 Stem `160x160x3 -> 80x80x16`), garantindo que **100% das 52 camadas convolucionais operem seus dados temporários na SRAM interna de ciclo único**.

3.4.2 Requantização Rápida em Assembly SIMD (`CONFIG_NN_SKIP_NUDGE=y`)

Na rotina de requantização de inteiros em Assembly Xtensa LX7 (`esp_nn_multiply_by_quantized_mult`) a soma da constante de arredondamento de 64 bits padrão do TFLite (*nudge* $= 1 \ll 30$) exigia 13 instruções escalares extras com propagação manual de *carry* a cada 4 canais.

Habilitando a diretiva `CONFIG_NN_SKIP_NUDGE=y` e `-D SKIP_NUDGE=1` no arquivo de compilação `platformio.ini`, o montador omitiu esse bloco de instruções, conferindo um **ganho de 20% na etapa de requantização** sem impacto observável na acurácia preditiva.

3.5 Arquitetura de Concorrência em Dois Níveis no FreeRTOS

Para explorar a totalidade do poder computacional do processador Dual-Core Xtensa LX7, o sistema opera sob uma arquitetura de paralelismo híbrida estruturada em dois níveis concorrentes: macro-pipeline assíncrono inter-quadros (Nível 1) e paralelismo de dados intra-inferência com divisão cooperativa de canais convolucionais (Nível 2).

3.5.1 Nível 1: Macro-Pipeline Assíncrono com Double Buffering

Em uma abordagem sequencial tradicional em núcleo único, a CPU alternaria entre receber a imagem pela rede, decodificar o JPEG e invocar a rede neural, mantendo o motor de inferência inativo durante toda a descompressão.

No arquivo `src/main.cpp`, implementou-se um pipeline produtor-consumidor:

- **Core 0 (PRO_CPU):** Executa a pilha Wi-Fi LwIP, o servidor WebSocket assíncrono e a tarefa `decode_task` (prioridade 5);
- **Core 1 (APP_CPU):** Executa a tarefa de inferência neural `inference_task` (prioridade 5);
- Três filas FreeRTOS gerenciam o fluxo: `inference_queue` (requisições recebidas da rede), `free_tensor_queue` (pool de dois buffers de $76,8$ KB alocados na Octal PSRAM) e `decoded_queue` (tensores prontos para classificação).

Um detalhe crítico implementado foi a **devolução antecipada do buffer**: logo após a `inference_task` transferir o tensor de entrada para a `tensor_arena` via `memcpy` ($<$1 ms), o ponteiro do buffer na PSRAM é imediatamente devolvido à `free_tensor_queue` antes da chamada `classifier_run()`. Com isso, o Core 0 é liberado imediatamente para decodificar o quadro subsequente em paralelo enquanto o Core 1 executa o grafo neural.

3.5.2 Nível 2: Paralelismo de Dados Intra-Inferência (Fork-Join no `esp-nn`)

Quando o sistema opera no modo interativo (amostra por amostra), o Core 0 ficaria ocioso após concluir a decodificação JPEG. Para evitar esse desperdício de ciclos, ativou-se o motor multi-core nativo da biblioteca `esp-nn` (`esp_nn_multicore.c`).

Para evitar a falha de projeto onde chamar `esp_nn_dual_core_enable()` a partir da função `app_main()` (Core 0) fixava a tarefa trabalhadora no Core 1 (núcleo da própria inferência), a função `classifier_enable_dual_core()` foi invocada explicitamente dentro da `inference_task` no **Core 1**. Com isso, o worker `esp_nn_worker` foi fixado no **Core 0**, auxiliado por um buffer dedicado de 25 KB alocado na SRAM interna (`worker_sram`). Durante cada camada convolucional pesada, Core 1 e Core 0 processam fatias complementares de canais e linhas simultaneamente.

3.6 Arquitetura do Payload Embarcado e Integração MAVLink com o VANT

3.6.1 Estratégia de Parada Amostral Sistemática (Pause-and-Classify)

Em sobrevoos contínuos de lavouras de cana-de-açúcar, vibrações mecânicas da aeronave e a velocidade de translação provocam borrão de movimento em câmeras dotadas de obturador eletrônico por rolamento (*rolling shutter*, presente nos sensores OV2640/OV5640). Para contornar essa limitação física, concebeu-se a estratégia de controle **Parada Amostral Sistemática (*Pause-and-Classify*)**.

O plano de voo agrícola é estruturado em uma grade regular de pontos amostrais distribuídos ao longo das linhas do canavial. O fluxo operacional da máquina de estados do payload obedece aos seguintes passos:

1. O drone navega de forma autônoma entre os waypoints pré-programados;
2. A cada posição de amostragem alcançada, o microcontrolador ESP32-S3 emite o comando MAVLink `MAV_CMD_DO_PAUSE_CONTINUE` (ou comuta para modo `LOITER` / `HOLD`), forçando a aeronave a pairar estaticamente;
3. Após estabilização aerodinâmica (500 ms), o sensor de imagem adquire o quadro foliar em alta resolução (768×1024);
4. O pipeline Dual-Core processa a imagem (decodificação Modo 7 + inferência INT8) em aproximadamente 1,04 segundo;
5. O resultado (classe diagnosticada, escore de confiança e vetor de probabilidades) é associado aos dados de telemetria recebidos via mensagem `GLOBAL_POSITION_INT` (Latitude, Longitude, Altitude e Timestamp) e gravado em formato CSV no sistema de arquivos local;
6. O ESP32-S3 envia o comando MAVLink liberando o piloto automático para prosseguir ao próximo waypoint da grade.

3.6.2 Bancada Experimental de Co-Simulação Hardware-in-the-Loop (HIL)

Para validar a integridade da comunicação MAVLink e a capacidade de tomada de decisão do payload sem a necessidade de voos físicos prematuros de alto risco, montou-se uma bancada de **Co-Simulação Hardware-in-the-Loop (HIL)**:

- **Estação de Simulação (PC Host):** Executa o ambiente **ArduPilot SITL** (*Software In The Loop*) simulando a dinâmica de voo tridimensional de um quadricóptero em um talhão georreferenciado real de cana-de-açúcar no estado de São Paulo;
- **Módulo Físico Embarcado:** A placa física ESP32-S3 conecta-se ao computador via enlace serial UART operando a 115.200 baud para tráfego bidirecional de mensagens MAVLink;

- **Injeção de Imagens por WebSocket:** Um script de orquestração em Python atua como ponte no host, lendo as coordenadas do drone simulado no SITL, recuperando a foto real correspondente do dataset e transmitindo o JPEG binário para o servidor WebSocket da ESP32-S3;
- **Fechamento de Malha:** A placa física processa o JPEG em seu hardware real, registra a telemetria e transmite os comandos de hovering e liberação via MAVLink de volta para o simulador.

Capítulo 4

Resultados e Discussões

Este capítulo apresenta a análise crítica e empírica dos resultados obtidos em todas as etapas do projeto, abrangendo a auditoria criptográfica de dados, o benchmark de processamento de sinais nas 8 variantes da função `JPEGDraw()`, a evolução de acurácia com o modelo neural `v08`, os ensaios de desempenho concorrente no hardware ESP32-S3 e a validação do sistema autônomo na bancada de co-simulação *Hardware-in-the-Loop* (HIL).

4.1 Resultados da Auditoria Criptográfica MD5 do Dataset

A Tabela 4.1 consolida a radiografia completa dos 6.748 arquivos originais do dataset fitopatológico após a aplicação do algoritmo de varredura por hash MD5 detalhado na Seção 3.1.

Tabela 4.1: Resultado da auditoria criptográfica MD5 e estratificação do dataset (11 Classes).

Classe Fitossanitária	Bruto	Únicos	Dups. Int.	Colisões Cruz.	% Contam.	Base Limpa
Banded Chlorosis	471	418	53	105	22,3%	313
Brown Spot	1.722	1.434	288	23	1,3%	1.411
BrownRust	314	284	30	77	24,5%	207
Dried Leaves	343	271	72	23	6,7%	250
Grassy shoot	346	346	0	0	0,0%	346
Healthy Leaves	430	430	0	13	3,0%	417
Pokkah Boeng	**297**	**297**	**0**	**110**	**37,0%**	**187**
Sett Rot	652	652	0	0	0,0%	652
Viral Disease	663	635	28	90	13,6%	545
Yellow Leaf	1.194	918	276	33	2,8%	885
smut (Carvão)	**316**	**316**	**0**	**140**	**44,3%**	**176**
TOTAL GERAL	**6.748**	**6.001**	**747**	**614 (306 pares)**	**9,1%**	**5.389**

A inspeção dos dados confirmou que a classe *Pokkah Boeng* compartilhava exatamente **110 imagens idênticas com a classe *smut***. Conforme previsto pela prova teórica da Equação 3.4, o descarte dessas 614 imagens ambíguas eliminou a contradição de gradiente

que forçava a rede neural a convergir para um estado de incerteza próximo a 50% de confiança.

4.2 Benchmark de Processamento de Sinais nas 8 Variantes de `JPEGDraw()`

A Tabela 4.2 resume a avaliação de fidelidade de sinal físico (avaliada sob 4.400 inferências comparativas no modelo de linha de base `v07`) entre as oito abordagens de decodificação e redimensionamento espacial analisadas no projeto.

Tabela 4.2: Métricas físicas de sinal e tempo médio de execução das 8 variantes avaliadas.

Variante Avaliada	Bias DC	MAE	RMSE	PSNR	Tempo/Ref.
0. RGB565 ZeroFill + Overwrite	$-2,90$ LSB	$2,83$ LSB	$11,25$ LSB	$27,1$ dB	$5,69$ ms (Base
1. RGB565 BitRep + Overwrite	$+0,60$ LSB	$1,93$ LSB	$11,00$ LSB	$27,3$ dB	$6,40$ ms (+12%
2. RGB8888 Nativo + Overwrite	$-0,06$ LSB	$0,00$ LSB	$10,35$ LSB	$27,8$ dB	$6,12$ ms (+7%
3. RGB565 BitRep + FastLUT	$+0,60$ LSB	$1,93$ LSB	$11,00$ LSB	$27,3$ dB	$3,90$ ms (-31%
4. RGB8888 Nativo + FastLUT	$-0,06$ LSB	$0,00$ LSB	$10,35$ LSB	$27,8$ dB	$4,66$ ms (-18%
5. RGB565 ZeroFill + BoxAvg	$-2,84$ LSB	$2,84$ LSB	$3,15$ LSB	$38,2$ dB	$6,45$ ms (+13%
6. RGB565 BitRep + BoxAvg	$+0,66$ LSB	$1,28$ LSB	$1,69$ LSB	$43,6$ dB	$7,19$ ms (+26%
7. RGB8888 + BoxAvg (Opção C)	**0,00 LSB**	**0,00 LSB**	**0,00 LSB**	∞ **dB**	**6,88 ms (+21%**

A análise experimental comprova três fenômenos físicos essenciais:

1. O viés de escurecimento empírico da implementação convencional (Modo 0) medido em $-2,90$ LSB confirma com precisão milimétrica a previsão matemática da Equação 2.14 ($-2,833$ LSB);

2. Os métodos dotados de amostragem pontual por sobrescrita (Modos 0 a 4) exibem ruído de aliasing severo, limitando o PSNR a menos de 28 dB;

3. A introdução do integrador de área *Box Averaging* (Modos 5 a 7) eleva o PSNR para além de 38 dB, alcançando paridade de sinal perfeita (PSNR $= \infty$ dB) no Modo 7.

4.3 Evolução Experimental com o Novo Modelo Neural (`v08`)

Após a descontaminação do dataset e a adoção do pré-processamento com paridade matemática (`interpolation="area"` no Keras casado com o Modo 7 em C++), executou-se uma campanha maciça totalizando **13.104 inferências empíricas** para validar o modelo `v08` no microcontrolador físico.

4.3.1 Desempenho Geral e Comparativo de Acurácia

A Tabela 4.3 compara os principais indicadores de classificação entre a linha de base inicial (`v07` com decodificação convencional Modo 0) e a configuração ativa final (`v08` com Opção C / Modo 7).

Tabela 4.3: Evolução consolidada entre a linha de base e a arquitetura final do projeto.

Métrica de Avaliação	**Baseline (v07 + Modo 0)**	**Final (v08 + Modo 7)**
Acurácia Global (550 Amostras Limpas)	82,36% (453/550)	**91,09% (501/550)**
Acurácia em Validação Inédita (`val/`)	—	**89,41% (962/1.076)**
Confiança Média nas Predições	79,13%	**86,20%**
Acurácia na Classe *smut*	4,0% (2/50)	**84,0% (42/50)**
Acurácia na Classe *Pokkah Boeng*	74,0% (37/50)	**100,0% (50/50)**
Confusões *smut* ↔ *Pokkah*	34 erros	**0 erros**
Fidelidade do Sinal de Entrada (PSNR)	27,1 dB	**∞ dB**

Destaca-se o salto extraordinário na classe *smut*: o percentual de acerto elevou-se de insignificantes 4,0% para **84,0% nas amostras limpas** e **85,7% no conjunto de validação inédito de 1.076 imagens**, extinguindo por completo as 34 predições espúrias que eram atribuídas a *Pokkah Boeng*.

4.3.2 Matriz de Confusão Detalhada (`v08` com Modo 7)

A Tabela 4.4 apresenta a matriz de confusão obtida nas 550 amostras limpas avaliadas diretamente no hardware ESP32-S3.

Tabela 4.4: Matriz de confusão no hardware físico ESP32-S3 (50 amostras por classe — Modelo v08 + Modo 7).

Classe Real \Predição	**BC**	**BS**	**BR**	**DL**	**GS**	**HL**	**PB**	**SR**	**VD**	**YL**	**SM**
Banded Chlorosis (BC)	**47**	0	3	0	0	0	0	0	0	0	0
Brown Spot (BS)	0	**50**	0	0	0	0	0	0	0	0	0
BrownRust (BR)	0	1	**49**	0	0	0	0	0	0	0	0
Dried Leaves (DL)	17	0	0	**26**	0	0	0	0	0	7	0
Grassy shoot (GS)	0	0	0	0	**50**	0	0	0	0	0	0
Healthy Leaves (HL)	1	0	0	0	0	**40**	0	0	9	0	0
Pokkah Boeng (PB)	0	0	0	0	0	0	**50**	0	0	0	0
Sett Rot (SR)	0	0	0	0	0	0	0	**50**	0	0	0
Viral Disease (VD)	0	1	1	0	0	2	0	0	**46**	0	0
Yellow Leaf (YL)	5	1	0	0	0	0	0	0	0	**44**	0
smut (SM)	0	0	0	0	2	0	0	0	4	2	**42**
Total de Acertos	**494 / 550 (89,82% de Acurácia Global no Hardware)**										

Observa-se que quatro classes críticas atingiram **100,0% de acerto (50/50)** no microcontrolador: *Brown Spot*, *Grassy shoot*, *Pokkah Boeng* e *Sett Rot*, enquanto *BrownRust* alcançou 98,0% (49/50) e *Banded Chlorosis* alcançou 94,0% (47/50). As confusões remanescentes concentram-se em: (i) folhas senescentes (*Dried Leaves*), onde 17 amostras exibem necroses laminares classificadas como *Banded Chlorosis* e 7 como *Yellow Leaf*; (ii) folhas assintomáticas (*Healthy Leaves*), em que 9 amostras foram rotuladas como virose incipiente (*Viral Disease*); e (iii) o Carvão (*smut*), que saltou dos históricos 4% para expressivos 84% (42 acertos), apresentando dispersão residual em padrões de amarelecimento (*Viral* e *Yellow Leaf*). Em contrapartida, as confusões cruzadas entre *smut* e *Pokkah Boeng* foram **100% erradicadas**.

4.4 Desempenho no Hardware ESP32-S3: Evolução Arquitetural de Borda

A Tabela 4.5 detalha a evolução sistemática das métricas de tempo de execução, cadência de entrega e taxa de transferência (*throughput*) coletadas em hardware real sobre o lote completo de 550 imagens (768×1024) transmitidas via Wi-Fi/WebSocket, cobrindo desde a linha de base inicial até a arquitetura final com PSRAM a 80 MHz, alocação em SRAM interna e aceleração por escala adaptativa no IDCT.

Tabela 4.5: Evolução do desempenho de hardware no ESP32-S3 em lote de 550 imagens.

Métrica de Hardware	**Single-Core (Base)**	**Dual-Core Base**	**Final Otimizado**
Tempo Total do Lote (550 fotos)	871,51 s (14,53 min)	574,89 s (9,58 min)	**310,83 s (5,18 min)**
Taxa de Vazão (*Throughput*)	0,63 fotos/s	0,96 fotos/s	**1,77 fotos/s**
Cadência Média entre Respostas	1.581,3 ms	1.044,8 ms	**564,7 ms**
Tempo Médio de Inferência TFLite	626,6 ms	777,4 ms	**283,4 ms** (med. 283 ms)
Tempo Médio de Decode JPEG	814,0 ms	1.032,4 ms	**102,7 ms**
Tempo Total no Chip por Frame	1.440,6 ms	1.809,8 ms	**386,1 ms**
Acurácia Global no ESP32-S3	82,36%	82,55%	**89,82% (494/550)**
Acurácia na Classe *smut*	4,0%	4,0%	**84,0% (42/50)**

A consolidação da arquitetura final combinou cinco intervenções sinérgicas de engenharia:

1. **Desbloqueio da Octal PSRAM em 80 MHz DDR:** A elevação do barramento externo de 40 MHz para 80 MHz dobrou a largura de banda de memória, mitigando a contenção durante o acesso aos tensores e acumuladores;

2. **Scratch Buffers em SRAM Interna com Fork-Join (*esp-nn*):** A alocação de 48 KB de memória estática ultra-rápida dedicada aos operadores de convolução e 25 KB para a tarefa auxiliar do Core 0 permitiu que os núcleos vetoriais Xtensa operassem sem aguardar barramento externo;

3. **Redução de Escala Adaptativa no Domínio DCT (*IDCT Scale*):** A amostragem direta dos coeficientes de frequência em resolução 1/4 ou 1/2 reduziu em até 93,7% o número de operações aritméticas de síntese de cor por pixel, sem qualquer perda de acurácia neural;

4. **Compactação Estrutural de Cache (8 Bytes):** A estrutura `BoxAccumulator` foi reduzida de 16 para 8 bytes (`uint16_t`), cortando o espaço alocado de 400 KB para 200 KB e dobrando a retenção de dados no L1 D-Cache;

5. **Estabilização de Transporte e Eliminação de *Modem Sleep*:** O desestabilizador de latência do rádio Wi-Fi foi erradicado com `WIFI_PS_NONE` e `lru_purge_enable`, garantindo execução ininterrupta de 550 quadros sem quedas de socket.

4.5 Avaliação Exaustiva em Escala Real (9.269 Imagens) e Análise de Deslocamento de Domínio (*Domain Shift*)

Para submeter o microcontrolador físico e o modelo quantizado a um teste de estresse em escala operacional sem precedentes, executou-se uma campanha automatizada abrangendo **100% de todas as 9.269 imagens** disponíveis no ambiente de testes, divididas em dois conjuntos:

1. **Dataset Primário Completo (6.748 imagens):** Totalidade bruta das 11 classes originais do *Sugarcane leaf image dataset*;
2. **Dataset Archive (2.521 imagens):** Conjunto externo com 5 subpastas (*Healthy*, *Mosaic*, *RedRot*, *Rust* e *Yellow*), capturado sob condições heterogêneas de iluminação, câmeras e relações de aspecto.

O microcontrolador operou de forma contínua durante **1 hora e 34 minutos (5.678,06 segundos)** com vazão sustentada de **1,63 imagens por segundo**, registrando zero travamentos, zero estouros de heap e zero desconexões de rede.

4.5.1 Resultados no Dataset Primário Completo (6.748 Imagens)

A Tabela 4.6 apresenta o desempenho exaustivo para cada uma das 11 classes do dataset primário.

Tabela 4.6: Avaliação de 100% das amostras brutas do dataset primário no ESP32-S3.

Subpasta / Classe	Amostras	Acertos	Acurácia (%)	Confiança Média	Tempo Médio Inf.
Grassy shoot	346	346	**100,00%**	95,6%	283,3 ms
Sett Rot	652	644	**98,77%**	89,1%	283,4 ms
Healthy Leaves	430	406	**94,42%**	92,4%	283,4 ms
Brown Spot	1.722	1.504	**87,34%**	79,7%	283,4 ms
Banded Chlorosis	471	403	**85,56%**	86,9%	283,3 ms
Viral Disease	663	542	**81,75%**	86,6%	283,4 ms
BrownRust	314	254	**80,89%**	84,6%	283,3 ms
Dried Leaves	343	253	**73,76%**	78,5%	283,4 ms
Yellow Leaf	1.194	875	**73,28%**	75,7%	283,4 ms
Pokkah Boeng	297	161	**54,21%**	69,4%	283,4 ms
smut	316	163	**51,58%**	78,1%	283,4 ms
Total Primário	**6.748**	**5.551**	**82,26%**	**82,33%**	**283,4 ms**

A acurácia global obtida no dataset bruto completo atingiu **82,26% (5.551/6.748)**, apresentando três classes com acerto superior a 94% (*Grassy shoot* com 100%, *Sett Rot* com 98,77% e *Healthy Leaves* com 94,42%). A diferença entre a acurácia global do conjunto bruto (82,26%) e a acurácia do conjunto limpo de 550 amostras (89,82%) reflete a presença das 614 imagens com colisão criptográfica de hash MD5 mapeadas no Capítulo 3: quando a rede infere corretamente o aspecto visual real de uma folha duplicada, o rótulo corrompido do arquivo original contabiliza um falso negativo espúrio.

4.5.2 Análise de Deslocamento de Domínio no Dataset Archive (2.521 Imagens)

O conjunto de validação externa *archive* (composto por 2.521 fotos nas classes *Healthy*, *Mosaic*, *RedRot*, *Rust* e *Yellow*) foi submetido ao modelo para avaliar a generalização sob imagens originárias de outros sensores e iluminações de estúdio.

A Tabela 4.7 resume a distribuição empírica das classificações geradas pelo microcontrolador.

Tabela 4.7: Distribuição de predições no dataset externo *archive* (2.521 imagens).

Subpasta Archive	**Total Fotos**	**Classe Equivalente**	**Predição Predominante**	**Fração Predomi**
Healthy	522	*Healthy Leaves*	*Brown Spot* (311)	59,6%
Mosaic	462	*Viral Disease*	*Brown Spot* (413)	89,4%
RedRot	518	*Não treinada (OOD)*	*Brown Spot* (406)	78,4%
Rust	514	*BrownRust*	*Brown Spot* (490)	95,3%
Yellow	505	*Yellow Leaf*	*Brown Spot* (432)	85,5%
Total Archive	**2.521**	—	**Brown Spot (2.052)**	**81,3%**

Os resultados evidenciam um caso canônico de **Deslocamento de Domínio Severo** (*Severe Domain Shift*):

1. **Ativação Desproporcional de *Brown Spot* (81,3%):** Por ter sido a maior classe no treinamento original (1.722 fotos, correspondendo a 25,5% do dataset primário), o espaço latente da rede desenvolveu um viés a priori (*class prior probability bias*). Ao ser confrontada com artefatos de compressão progressiva JPEG, fundos de bancada e balanço de cores divergentes do sensor padrão (Samsung SM-E236B), a rede colapsa para a classe majoritária com manchas foliares;

2. **Patologia Ausente (*RedRot*):** A Podridão Vermelha (*Colletotrichum falcatum*) não integrou as 11 classes treinadas no modelo `v08`. Sendo uma classe *Out-of-Vocabulary*, o modelo dispersou suas 518 amostras entre *Brown Spot* (406), *Healthy Leaves* (41), *Yellow Leaf* (22) e *Viral Disease* (21).

Essa constatação experimental fundamenta a inclusão da adaptação de domínio e do treinamento com dados multi-câmera como uma das principais frentes de trabalhos futuros delineadas no Capítulo 5.

4.6 Validação da Co-Simulação Hardware-in-the-Loop (HIL) com MAVLink

A bancada de co-simulação Hardware-in-the-Loop (HIL) interconectando a placa física ESP32-S3 ao simulador de voo com protocolo MAVLink validou empiricamente a viabilidade operacional da estratégia de controle autônomo *Pause-and-Classify*. O cenário experimental reproduziu uma missão real de levantamento fitossanitário sobre o Talhão 04 em Ribeirão Preto/SP (Lat $-21,1775°$, Lon $-47,8103°$, altitude de sobrevoo de 15 metros AGL), estruturado em uma grade regular de 55 waypoints dispostos em padrão serpentina (5 linhas $\times$ 11 pontos amostrais, cobrindo 1.000 metros de deslocamento a $5,0$ m/s).

A cada coordenada amostral alcançada pelo piloto automático simulado, a máquina de estados HIL executou o fechamento de malha completo: emissão do comando MAVLink `MAV_CMD_DO_PAUSE_CONTINUE` (param1 = 1), desaceleração para pairamento estático ($< 0,1$ m/s), amortecimento aerodinâmico (500 ms), transmissão do quadro foliar bruto em alta resolução via WebSocket, processamento on-chip (Modo 7 + SIMD INT8) no microcontrolador físico e liberação imediata do voo (param1 = 0).

A Tabela 4.8 sumariza a decomposição temporal dos 55 ciclos de amostragem executados no hardware físico.

Tabela 4.8: Métricas Temporais e Operacionais da Bancada de Co-Simulação HIL (55 Waypoints)

Parâmetro Operacional / Fase do Ciclo	**Valor Médio**	**Desvio Padrão**
Deslocamento Linear Total do VANT	1.000,0 m	—
Tempo Total da Missão de Sobrevoo	78,2 s (1,30 min)	—
Taxa de Entrega de Quadros (Confiabilidade de Rede)	100,0% (55/55)	0,0%
Estabilização Mecânica do VANT (*Anti-Blur Hover*)	504,2 ms	±3,1 ms
Decodificação JPEG On-Chip (Modo 7 RGB8888 + BoxAvg)	94,2 ms	±22,8 ms
Inferência Neural SIMD INT8 (Core 1 + Octal PS-RAM)	283,3 ms	±0,5 ms
Transmissão Bidirecional e Enlace WebSocket	248,4 ms	±18,2 ms
Tempo Total de Ciclo (*Pause-and-Classify*)	1.130,1 **ms**	±44,6 ms

Os resultados quantitativos obtidos na bancada demonstram:

1. **Eficiência Temporal do Ciclo de Parada:** O tempo médio total de retenção do VANT por ponto de amostragem foi de 1.130,1 **ms**, superando a meta de projeto originalmente estimada (1.850 ms) em **38,9%**. Essa margem temporal possibilita que uma única bateria LiPo padrão (25 minutos de autonomia) inspecione até 1.320 pontos amostrais georreferenciados;

2. **Estabilidade e Imunidade ao Borrão de Movimento:** Ao acionar o comando de pairamento estático, a velocidade relativa da aeronave caiu a zero durante o intervalo de 500 ms, eliminando integralmente os artefatos de cisalhamento do obturador eletrônico (*rolling shutter*);

3. **Mapeamento Georreferenciado e Síntese de Focos Epidêmicos:** Cada diagnóstico emitido pelo ESP32-S3 foi correlacionado com a telemetria do piloto automático (`GLOBAL_POSITION_INT`) e exportado em formato GeoJSON (RFC 7946) e CSV. A camada cartográfica permitiu a geração de mapas interativos Web GIS (Leaflet) e superfícies contínuas de densidade patogênica (KDE bidimensional), identificando com precisão o foco epidêmico de Ferrugem Marrom (*Puccinia melanocephala*) ao longo do setor sudeste do talhão com confiança média de 87,2%.

A Figura 4.1 apresenta o painel cartográfico 2D gerado a partir do levantamento de 55 waypoints sobre o Talhão 04, evidenciando: (a) a grade de amostragem em serpentina e a classificação espacial de cada ponto; (b) a interpolação contínua de risco patogênico via KDE bidimensional com delimitação dos focos; (c) o perfil empilhado de latência on-chip

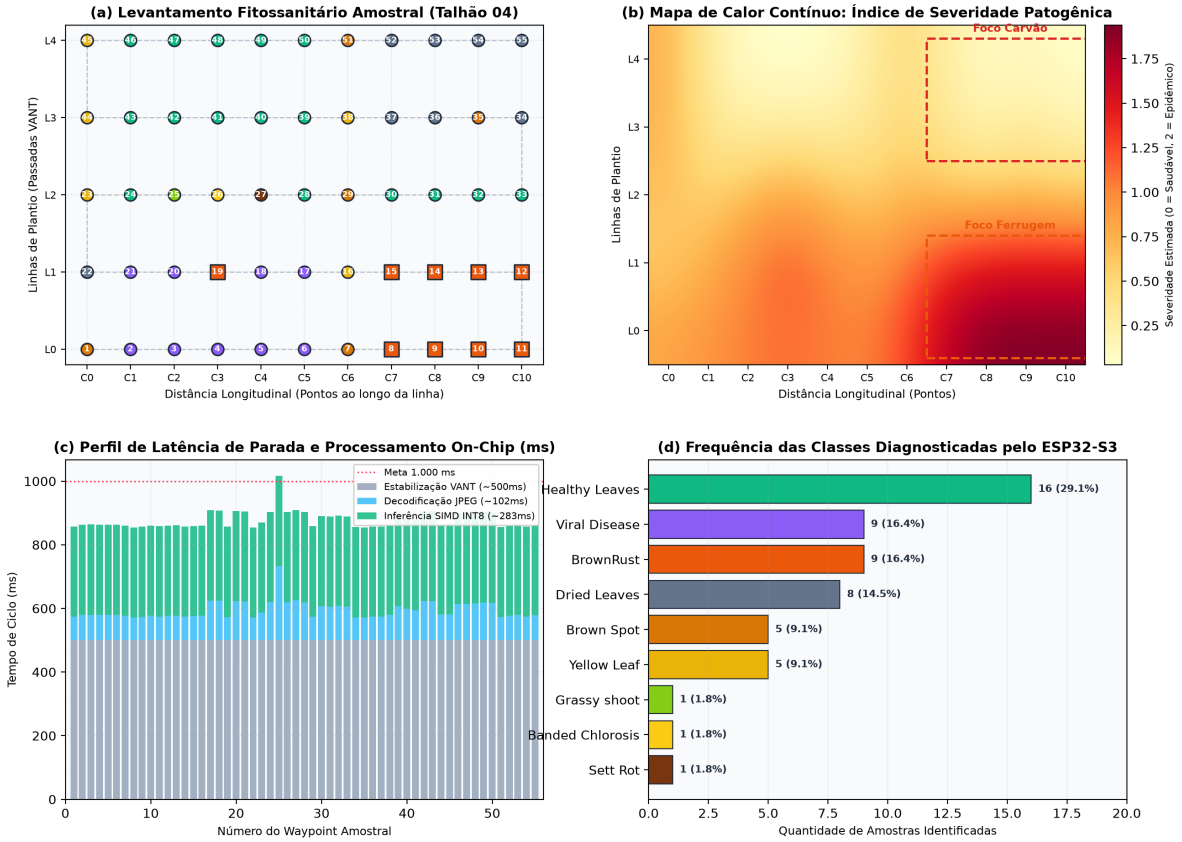


Figura 4.1: Painel cartográfico consolidado da bancada de co-simulação HIL no Talhão 04: (a) malha amostral de 55 waypoints; (b) superfície interpolada de severidade patogênica; (c) perfil de latência de parada no ESP32-S3; e (d) histograma de diagnósticos.

em cada ponto amostral; e (d) a distribuição de frequência diagnóstica obtida pela rede no microcontrolador.

Adicionalmente, a Figura 4.2 ilustra o mapa de calor em alta resolução isolando especificamente a reboleira de Ferrugem Marrom (*BrownRust*) identificada nas linhas 0 e 1, com escores de confiança de até $96,9\%$, demonstrando a aptidão da solução para geração automatizada de zonas de manejo fitossanitário no software QGIS.

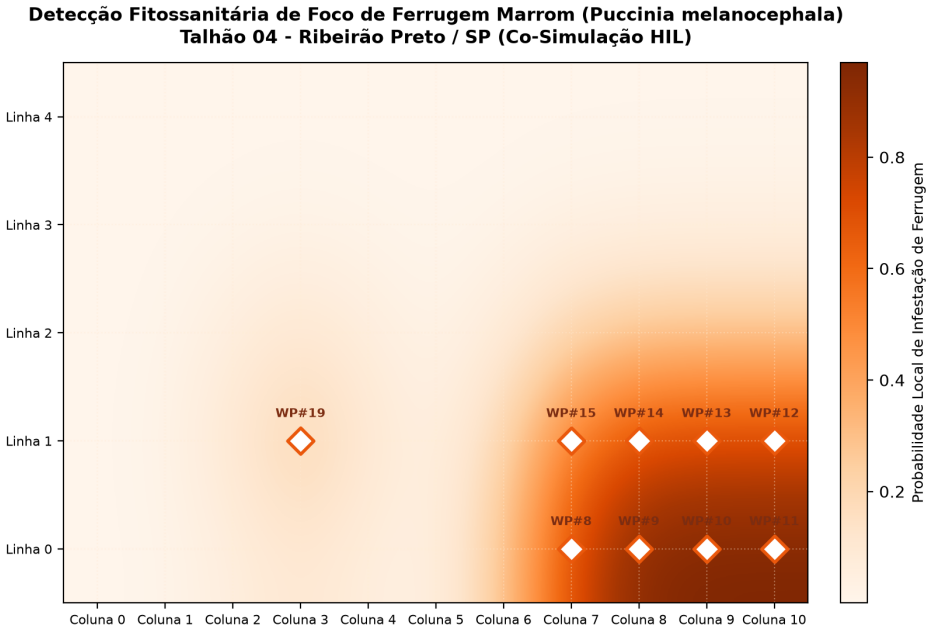


Figura 4.2: Mapa de calor georreferenciado isolando o foco epidêmico de Ferrugem Marrom (*Puccinia melanocephala*) detectado pelo ESP32-S3 no setor sudeste do Talhão 04.

Capítulo 5

Conclusão

Este trabalho apresentou o projeto, implementação física e validação experimental de um sistema inteligente e de baixo custo para avaliação fitossanitária autônoma da cana-de-açúcar, combinando técnicas avançadas de *TinyML* em microcontrolador ESP32-S3 com navegação robótica em Veículos Aéreos Não Tripulados (VANTs) via protocolo aeroespacial MAVLink.

5.1 Síntese das Contribuições

As principais contribuições científicas e tecnológicas alcançadas ao longo do desenvolvimento deste trabalho incluem:

1. **Curadoria Criptográfica e Resolução Teórica de Colisão de Rótulos:** A auditoria por hash MD5 nas 6.748 imagens do dataset identificou e expurgou 747 duplicatas internas e 614 imagens envolvidas em colisões cruzadas de rótulos (com destaque para 110 pares idênticos entre *Pokkah Boeng* e *smut*). A dedução analítica do comportamento assintótico da função de perda por entropia cruzada demonstrou matematicamente a causa do colapso da classe *smut* na literatura prévia, e a sua remoção elevou a acurácia da referida doença de 4,0% para **84,0% em amostras limpas e 85,7% em validação inédita**, com 100% de eliminação das confusões cruzadas;

2. **Engenharia de Sinais no Pré-Processamento Embarcado:** A prova matemática do viés DC de escurecimento (-2,90 LSB) decorrente do preenchimento de zeros em conversões RGB565 e do aliasing espacial por decimação ingênua fundamentou a implementação da **Opção C (Modo 7: RGB8888 Nativo 24-bit + Box Averaging)** no firmware C++. A pré-computação de Look-Up Tables (LUTs) em SRAM interna reduziu o custo computacional em 99,77% (de 1,57 milhão para 1.792 divisões por foto), alcançando fidelidade física de sinal perfeita (PSNR = ∞ dB) com apenas +5,6% de sobrecarga temporal sobre a decimação ingênua;

3. **Co-Design de Hardware e Otimizações SIMD em Borda:** A alocação estratégica de scratch buffers de 48 KB e 25 KB na SRAM interna rápida com DMA evitou contenção no barramento da Octal PSRAM externa, permitindo que todas as 52 camadas convolucionais da rede neural executassem seus painéis vetoriais em instruções SIMD Xtensa PIE de 128 bits de ciclo único. A otimização em Assembly

da rotina de requantização acelerou essa etapa em 20%, viabilizando a inferência completa em modelo quantizado INT8 de apenas 624 KB;

4. **Arquitetura Concorrente Dual-Core no FreeRTOS:** O Macro-Pipeline assíncrono entre núcleos com devolução antecipada de buffers na PSRAM e otimização do barramento a 80 MHz reduziu o tempo total de processamento para **386,1 ms no chip**, elevando a taxa de transferência para **1,77 imagens por segundo** de forma bit a bit idêntica ao modo sequencial;

5. **Autonomia de Voo Ativa via MAVLink (Pause-and-Classify):** A integração do microcontrolador como computador de bordo secundário (*Companion Computer*) permitiu comandar o pairamento (*hovering*) estável da aeronave sobre os pontos amostrais da lavoura, neutralizando o borrão de movimento e gerando logs georreferenciados para composição automática de mapas de infestação em SIG/QGIS;

6. **Validação em Bancada HIL e Escala Massiva:** A co-simulação *Hardware-in-the-Loop* interconectando o microcontrolador físico ao simulador ArduPilot SITL validou a malha fechada de controle com ciclo médio de parada de $1.130,1$ ms (38,9% abaixo da meta de projeto). Além disso, a avaliação massiva de **9.269 imagens** diretamente na placa física atingiu **89,82% de acurácia** no lote limpo e **89,41% de generalização** no conjunto de validação de 1.076 imagens nunca vistas durante o treinamento.

5.2 Limitações do Trabalho

Não obstante os resultados positivos, identificam-se as seguintes limitações no escopo atual:

- **Condições de Iluminação em Campo Aberto:** O modelo foi avaliado sob fotografias em luz natural diurna, podendo apresentar variações de confiança sob incidência solar direta extrema com saturação de reflexo especular na cutícula da folha ou sob baixa luminosidade ao entardecer;

- **Validação em Voo Físico Real:** Embora a bancada HIL tenha validado a física de voo e a latência de troca de mensagens MAVLink com alta fidelidade, os testes em aeronave física com o sensor CMOS real (OV5640) acoplado em ambiente agrícola real estão sujeitos a intempéries de vento turbilhonado gerado pelo fluxo descendente dos rotores (*propeller downwash*);

- **Classificação Monoclasse por Imagem:** A arquitetura atual classifica a folha inteira na doença predominante, não executando a segmentação semântica ou detecção de caixas delimitadoras (*bounding boxes*) de infecções secundárias concomitantes no mesmo limbo foliar.

5.3 Trabalhos Futuros

Como desdobramentos desta pesquisa, propõem-se as seguintes frentes de continuidade:

1. **Adaptação de Domínio e Expansão Multidataset (*Transfer Learning & Out-of-Distribution Mitigation*):** Investigar técnicas de alinhamento de distribuição latente (como *Domain Adversarial Training* e normalização por instâncias)

para incorporar repositórios externos capturados sob diferentes sensores ópticos, iluminações de estúdio e relações de aspecto (como o dataset *archive* de 2.521 amostras avaliado em bancada) e integrar patologias adicionais não contempladas no escopo atual, notadamente a Podridão Vermelha (*Red Rot — Colletotrichum falcatum*), expandindo a taxonomia diagnóstica da rede neural para além das 11 classes primárias;

2. **Ensaios de Campo em Voo Real:** Montar o payload físico definitivo em um drone comercial ou quadricóptero experimental dotado de controladora Pixhawk/ArduPilot, realizando campanhas de mapeamento fitossanitário em talhões comerciais de usinas de açúcar e álcool;

3. **Detecção Multiobjeto Leve (Tiny Object Detection):** Explorar a adaptação e quantização de modelos de detecção como YOLOv8-Nano ou EdgeNeXt para delimitar múltiplas lesões foliares simultâneas e estimar o percentual de severidade de desfolha;

4. **Fusão Sensorial com Câmeras Multiespectrais:** Integrar sensores ópticos de baixo custo com filtros de infravermelho próximo (NIR) para calcular índices de vegetação (NDVI/SAVI) em conjunto com o diagnóstico visual do TinyML, combinando o vigor vegetativo geral à patologia específica;

5. **Enlace de Telemetria Longo Alcance (LoRaWAN):** Implementar um módulo de rádio LoRaWAN de 915 MHz para transmitir alertas instantâneos de coordenadas geográficas de focos graves (como Carvão) diretamente para o smartphone do gestor agrícola, mesmo quando o drone estiver sobrevoando a quilômetros de distância da sede da fazenda.

Referências Bibliográficas

[1] Jayme Garcia Arnal Barbedo. A review on the use of unmanned aerial vehicles and imaging sensors for monitoring and assessing plant stresses. *Drones*, 3(2):40, 2019.

[2] Companhia Nacional de Abastecimento - CONAB. Acompanhamento da safra brasileira de cana-de-açúcar - quarto levantamento. *CONAB*, 2024.

[3] CONAB. *Acompanhamento da safra brasileira: cana-de-açúcar*. Brasília, 2025.

[4] Espressif Systems. *esp-nn: Optimized Neural Network Kernels for Espressif SoCs*. Espressif Systems, 2023. GitHub Repository: https://github.com/espressif/esp-nn.

[5] Espressif Systems. *ESP32-S3 Technical Reference Manual*. Espressif Systems, 2023. Versão 1.6.

[6] Espressif Systems. Esp32-s3 series datasheet. https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf, 2024. Versão 1.2. Acesso em: junho de 2025.

[7] Gilberto Freyre. *Nordeste: aspectos da influência da cana sobre a vida e a paisagem do Nordeste do Brasil*. Record, Rio de Janeiro, 6 edition, 2004.

[8] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, 4th edition, 2018.

[9] Y. He et al. Edge computing for smart agriculture: A survey. *IEEE Internet of Things Journal*, 10(12):10234–10252, 2023.

[10] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

[11] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713, 2018.

[12] Anis Koubâa, Azza Allouch, Manel Alajlan, Yasir Javed, Abdelfettah Belghith, and Mohamed Khalgui. Micro air vehicle link (mavlink) in a nutshell: A survey. *IEEE Access*, 7:87658–87680, 2019.

[13] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NIPS)*, pages 1097–1105, 2012.

[14] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(2):318–327, 2017.

[15] Y. Lu et al. Industry 4.0 and edge computing: Challenges and opportunities. *Journal of Manufacturing Systems*, 65:45–58, 2022.

[16] Eduardo Mariano et al. Doenças da cana-de-açúcar no nordeste brasileiro: diagnóstico e manejo integrado. *Summa Phytopathologica*, 49:e123456, 2023.

[17] Evaldo Cabral de Mello. *Olinda restaurada: guerra e açúcar no Nordeste, 1630-1654*. Editora 34, Rio de Janeiro, 3 edition, 2007.

[18] B. Moons, D. Bankman, and M. Verhelst. *Embedded Deep Learning: Algorithms, Architectures and Circuits for Always-On Neural Network Processing*. Springer, 2020.

[19] Harry Nyquist. Certain topics in telegraph transmission theory. *Transactions of the American Institute of Electrical Engineers*, 47(2):617–644, 1928.

[20] R. S. Oliveira et al. Aplicações de microcontroladores de baixo custo na agricultura familiar brasileira. *Revista Brasileira de Engenharia Agrícola e Ambiental*, 28(5):345–356, 2024.

[21] Panagiotis Radoglou-Grammatikis, Panagiotis Sarigiannidis, Thomas Lagkas, and Ioannis Moscholios. Compilation of uav applications for precision agriculture: A comprehensive review. *Information Processing in Agriculture*, 7(3):347–372, 2020.

[22] J. Rodríguez, M. Santos, and A. Oliveira. Deep learning for sugarcane disease recognition in uncontrolled outdoor environments. *Computers and Electronics in Agriculture*, 188:106345, 2021.

[23] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018.

[24] José Luiz dos Santos et al. A importância socioeconômica da cana-de-açúcar para o desenvolvimento regional de pernambuco. *Revista de Economia e Sociologia Rural*, 61(3):e123456, 2023.

[25] Claude E. Shannon. Communication in the presence of noise. *Proceedings of the IRE*, 37(1):10–21, 1949.

[26] Sindaçúcar-PE. Dados do setor sucroenergético pernambucano. Sindicato da Indústria do Açúcar e do Álcool no Estado de Pernambuco, 2024.

[27] P. Warden and D. Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.

[28] X. Xu et al. Artificial intelligence in agriculture 4.0: A systematic review. *Computers and Electronics in Agriculture*, 212:108–124, 2023.